VittaCare

Arquitetura – Bloco 3

Sumário

1. ARQUITETURA DE SOFTWARE 1
1.1. Visão geral da arquitetura 1
1.2. Estilo arquitetural 2
1.2.1. Organização em camadas 2
1.3. Módulos do sistema 3
1.4. Comunicação entre os componentes 4
1.5. Segurança e controle de acesso 4
1.5.1. Isolamento dos dados das clínicas 5
1.5.2. Medidas complementares de segurança 5
1.6. Stack tecnológica 6
1.7. Padrões de design 12
1.8. Modelo C4 13
1.8.1. C1 — Diagrama de contexto 13
1.8.2. C2 — Diagrama de containers 14
1.8.3. C3 — Diagrama de componentes 16
1.8.4. C4 — Diagrama de código 17
1.9. Architecture Decision Records (ADRs) 18
1.10. Considerações finais 22

1. ARQUITETURA DE SOFTWARE

1.1. Visão geral da arquitetura

O VittaCare será desenvolvido como uma aplicação web no modelo SaaS (Software as a Service), destinada ao gerenciamento de clínicas e unidades ambulatoriais. A plataforma permitirá o gerenciamento de pacientes, profissionais de saúde, especialidades, disponibilidades, agendamentos, atendimentos e relatórios administrativos.

A arquitetura será organizada no estilo monólito modular, no qual os módulos do sistema serão executados em uma única aplicação back-end, mantendo responsabilidades delimitadas e comunicação interna por interfaces e serviços. Essa abordagem preserva a simplicidade operacional necessária à versão inicial, sem impedir a evolução futura de módulos específicos.

A solução será dividida em três partes principais:

Front-end: interface de usuário desenvolvida com React.

Back-end: aplicação Java com Spring Boot, responsável pelas regras de negócio, autenticação, autorização, validações e disponibilização dos serviços.

Banco de dados: PostgreSQL, disponibilizado por meio do Amazon RDS, responsável pelo armazenamento persistente das informações.

A comunicação entre o front-end e o back-end ocorrerá por meio de API REST, utilizando JSON sobre HTTPS. A infraestrutura será hospedada na Amazon Web Services (AWS), com duas instâncias EC2 de aplicação distribuídas atrás de um Application Load Balancer (ALB), além de banco de dados RDS PostgreSQL em configuração Multi-AZ.

1.2. Estilo arquitetural

O estilo arquitetural escolhido para o VittaCare é o monólito modular com arquitetura em camadas. Nesse modelo, o back-end é implantado como uma aplicação única, mas seu código é dividido em módulos de domínio e camadas técnicas com responsabilidades específicas.

A escolha é adequada ao escopo inicial porque permite desenvolver, testar e implantar a solução com menor complexidade operacional do que uma arquitetura distribuída em microsserviços. Os módulos não deverão acessar diretamente os repositórios uns dos outros sem passar pelas interfaces ou serviços definidos, preservando o baixo acoplamento.

1.2.1. Organização em camadas

Camada de apresentação (Front-end): desenvolvida com React, apresenta as telas e permite a interação dos pacientes, profissionais de saúde, recepcionistas e administradores.

Camada de controle (Controllers): recebe requisições HTTP, aplica validações de entrada e encaminha as operações para os serviços correspondentes.

Camada de serviços (Services): concentra as regras de negócio, incluindo validação de horários, agendamentos, cancelamentos, alterações de disponibilidade, permissões e isolamento por clínica.

Camada de persistência (Repositories): realiza o acesso aos dados por meio de Spring Data JPA e Hibernate, aplicando consultas e operações transacionais.

Camada de dados (Database): utiliza PostgreSQL para armazenar clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades, agendamentos, atendimentos e registros de auditoria.

1.3. Módulos do sistema

Módulo	Responsabilidade
Autenticação	Realizar login, validação de credenciais, emissão e validação de tokens JWT.
Usuários e permissões	Gerenciar contas, perfis de acesso, papéis e permissões.
Clínicas	Gerenciar organizações cadastradas e o contexto de isolamento de cada clínica.
Pacientes	Cadastrar, consultar e atualizar informações dos pacientes.
Profissionais	Gerenciar dados dos profissionais de saúde.
Especialidades	Cadastrar especialidades e associá-las aos profissionais.
Disponibilidade	Gerenciar horários disponíveis e bloqueados dos profissionais.
Agendamentos	Criar, consultar, confirmar, cancelar e reagendar consultas, impedindo conflitos.
Atendimentos	Registrar comparecimento e ausência relacionados aos agendamentos.
Notificações	Enviar confirmações e lembretes relacionados aos agendamentos.
Relatórios e auditoria	Disponibilizar informações administrativas e registrar operações críticas.

Os módulos serão organizados dentro da mesma aplicação Spring Boot. A separação lógica não representa serviços independentes: todos compartilham o processo de execução do monólito, a configuração de segurança e a infraestrutura de persistência.

1.4. Comunicação entre os componentes

A comunicação entre o front-end e o back-end será realizada por meio de API REST, utilizando HTTPS e mensagens JSON. O fluxo básico será:

O usuário realiza uma ação na interface React.

O front-end envia uma requisição HTTPS para o endpoint correspondente.

O Controller recebe a requisição, valida os dados de entrada e encaminha a operação.

O Service verifica as regras de negócio, as permissões e o contexto da clínica associado ao usuário autenticado.

Quando necessário, o Service solicita operações ao Repository.

O Repository realiza a consulta ou alteração no PostgreSQL por meio do Spring Data JPA/Hibernate.

O back-end retorna uma resposta HTTP com os dados ou mensagem de erro em JSON.

O front-end apresenta o resultado ao usuário.

As operações de agendamento e bloqueio de disponibilidade deverão utilizar transações e restrições de integridade no banco de dados para evitar conflitos em solicitações concorrentes. A documentação dos endpoints será mantida com Swagger e OpenAPI.

1.5. Segurança e controle de acesso

Como o VittaCare armazenará dados pessoais e informações relacionadas à prestação de serviços de saúde, a arquitetura deverá aplicar autenticação, autorização, isolamento entre clínicas, proteção das comunicações e auditoria das operações relevantes.

A autenticação será realizada com Spring Security e JWT. Após validar as credenciais, o back-end emitirá um token assinado, com validade definida, que será enviado nas requisições subsequentes. O sistema deverá validar assinatura, expiração e permissões antes de permitir o acesso a recursos protegidos.

Paciente: poderá consultar e atualizar os próprios dados permitidos e realizar operações relacionadas aos seus agendamentos.

Profissional de saúde: poderá gerenciar sua disponibilidade e consultar sua agenda e os atendimentos autorizados.

Recepcionista: poderá cadastrar pacientes e realizar operações administrativas de agendamento conforme as permissões atribuídas.

Administrador: poderá gerenciar usuários, profissionais, especialidades, permissões e relatórios da clínica.

As permissões serão verificadas no back-end, em todos os endpoints protegidos. A interface não será considerada mecanismo de segurança: ocultar uma opção de menu não substitui a autorização no servidor.

1.5.1. Isolamento dos dados das clínicas

Por ser uma plataforma SaaS destinada a diferentes clínicas, o VittaCare adotará isolamento lógico dos dados. Cada registro pertencente a uma organização deverá estar associado a uma clínica por meio do identificador clinica_id, incluindo usuários, pacientes, profissionais, especialidades, disponibilidades, agendamentos, atendimentos e registros administrativos aplicáveis.

O back-end obterá o contexto da clínica a partir da identidade autenticada e das associações autorizadas do usuário. O clinica_id eventualmente enviado pelo front-end será tratado apenas como dado de entrada e nunca como prova de autorização. Antes de consultar, inserir, atualizar ou excluir registros, os serviços deverão validar se o registro pertence à clínica do contexto autenticado.

As consultas e operações de persistência deverão aplicar filtros por clinica_id. Chaves estrangeiras e restrições compostas deverão ser utilizadas, quando aplicável, para impedir associações entre registros de clínicas diferentes. Serão realizados testes automatizados de acesso cruzado entre clínicas.

1.5.2. Medidas complementares de segurança

Utilização obrigatória de HTTPS com TLS 1.2 ou superior entre cliente e serviços públicos.

Armazenamento de senhas com algoritmo próprio para hash de senhas, preferencialmente Argon2id ou BCrypt, com parâmetros adequados.

Validação de entradas, limites de tamanho e tratamento seguro de erros na API.

Autenticação e autorização em 100% dos endpoints que exponham informações restritas.

Proteção de credenciais e configurações sensíveis por mecanismo de gerenciamento de segredos; não armazenar segredos no repositório Git.

Restrição de acesso ao banco de dados em sub-redes privadas, permitindo conexão apenas a partir das instâncias de aplicação autorizadas.

Registro das operações administrativas críticas, incluindo usuário responsável, clínica, ação, entidade afetada e data/hora.

Aplicação de controles contra vulnerabilidades comuns, incluindo validação de entrada, proteção contra injeção, controle de sessão/token e configuração segura dos componentes.

1.6. Stack tecnológica

A stack tecnológica foi definida considerando os requisitos funcionais e não funcionais do VittaCare, especialmente a necessidade de interface web, API segura, integridade relacional dos agendamentos, isolamento de dados entre clínicas, disponibilidade mensal mínima de 99,9%, suporte à carga inicial projetada e recuperação de dados.

Camada	Tecnologia escolhida	Alternativa descartada	Justificativa técnica
Front-end	React	Angular	A interface será composta por telas e componentes reutilizáveis para os quatro perfis de usuário. React permite organizar componentes e integrar-se à API REST sem impor uma estrutura de servidor ao front-end.
Linguagem do back-end	Java	JavaScript com Node.js	Java oferece tipagem estática e recursos consolidados para estruturar regras de negócio, validações e operações transacionais

			necessárias aos agendamentos.
Framework do back-end	Spring Boot	Express.js	Spring Boot integra recursos para construção de APIs, injeção de dependência, validação, segurança e persistência, atendendo à organização em camadas proposta.
Comunicação	API REST com JSON sobre HTTPS	GraphQL	As operações previstas são recursos e ações bem delimitados, como cadastro, consulta e agendamento. REST permite contratos HTTP objetivos e integração simples com o front-end.
Banco de dados	PostgreSQL no Amazon RDS Multi-AZ	MySQL ou MongoDB	O modelo relacional atende às relações entre clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades e agendamentos, permitindo FKs, constraints e transações para

			preservar integridade. O PostgreSQL oferece índice único parcial nativo, usado para impedir mais de um agendamento ativo por disponibilidade, recurso que o MySQL não oferece.
Persistência	Spring Data JPA / Hibernate	JDBC puro	O mapeamento objeto-relacional reduz código repetitivo nas operações CRUD e integra-se aos repositories. Consultas específicas poderão usar JPQL ou SQL quando necessário.
Autenticação e autorização	Spring Security + JWT	Autenticação implementada manualmente	Centraliza filtros de segurança e autorização no back-end, permitindo proteger endpoints conforme identidade, perfil e contexto da clínica.
Documentação da API	Swagger / OpenAPI	Documentação exclusivamente manual	Define um contrato padronizado de

			endpoints, parâmetros, respostas e autenticação, facilitando integração e testes.
Versionamento	Git e GitHub	Compartilhamento manual de arquivos	Mantém histórico, branches, revisão de alterações e colaboração entre integrantes.
Containerização	Docker	Configuração manual de ambientes	Padroniza a execução da aplicação e reduz diferenças entre desenvolvimento, homologação e produção.
Hospedagem da aplicação	Amazon EC2 em duas instâncias atrás de ALB	Uma única instância ou servidor físico	Duas instâncias permitem distribuir requisições e manter capacidade de atendimento caso uma instância falhe. O ALB executa health checks e deixa de encaminhar tráfego a destinos não saudáveis.
Balanceamento	AWS Application Load Balancer	Distribuição manual de tráfego	Distribui requisições HTTPS entre as instâncias EC2 e permite verificações de

			integridade dos destinos.
Banco de dados gerenciado	Amazon RDS PostgreSQL Multi-AZ	Banco instalado manualmente em EC2	A configuração Multi-AZ oferece redundância gerenciada do banco, alinhada ao requisito de disponibilidade. A instância e o armazenamento deverão ser dimensionados e monitorados conforme carga real.
Armazenamento de arquivos	Amazon S3, quando necessário	Armazenamento no disco da aplicação	Mantém arquivos separados das instâncias de aplicação e permite acesso independente do ciclo de vida dos containers.
Monitoramento	Amazon CloudWatch	Acompanhamento exclusivamente manual	Permite coletar métricas, logs e alarmes para acompanhar erros, desempenho e disponibilidade.
Backup e recuperação	Backups automatizados do RDS com recuperação point-in-time (PITR) e AWS Backup para	Cópias manuais sem política centralizada	PITR e logs de transação dão suporte a RPO de até 15 minutos; cópias diárias serão mantidas por pelo menos 7 dias e snapshots

	cópias semanais		semanais por pelo menos 4 semanas. O RTO de até 4 horas deverá ser validado em testes periódicos de restauração.
Rede e proteção	AWS VPC, sub-redes privadas e Security Groups	Exposição direta dos serviços à internet	Restringe a comunicação entre balanceador, aplicação e banco, reduzindo a superfície de exposição.
Distribuição de conteúdo	Amazon CloudFront	Entrega de arquivos estáticos diretamente pela aplicação	Pode distribuir conteúdo estático e arquivos do S3, reduzindo latência para esse tipo de recurso.
Testes	: JUnit 5 e Mockito (back-end); Jest e React Testing Library (front-end)	Testes exclusivamente manuais	Permitem verificar regras de negócio, endpoints, autorização e regressões de forma repetível; o JUnit acompanha o starter de testes do Spring Boot e o Jest cobre os componentes React..

A infraestrutura de produção prevista considera duas instâncias EC2 com configuração inicial de 2 vCPUs e 2 GB de memória cada, distribuídas atrás de um Application Load Balancer. O banco de dados será Amazon RDS PostgreSQL com

implantação Multi-AZ e armazenamento inicial estimado de 50 GB gp3. Os valores de capacidade são premissas de dimensionamento inicial e deverão ser validados por testes de carga antes da entrada em produção.

Os ambientes DEV, STAGING e PROD deverão ser separados. Desenvolvimento e homologação utilizarão recursos e bases de dados próprios, sem compartilhar dados reais de produção. O acesso ao banco de produção será restrito às instâncias autorizadas da aplicação.

1.7. Padrões de design

Os padrões de design serão utilizados para organizar o código, separar responsabilidades e facilitar a manutenção e os testes do VittaCare. A aplicação dos padrões deverá permanecer compatível com o monólito modular, evitando complexidade desnecessária.

Padrão	Onde será aplicado	Motivo da utilização
Repository	Camada de persistência dos módulos de pacientes, profissionais, disponibilidade e agendamentos.	Isolar o acesso ao banco de dados das regras de negócio e concentrar consultas e operações de persistência.
Service Layer	Serviços de autenticação, agendamento, disponibilidade, usuários e clínicas.	Centralizar regras de negócio, validações e transações, evitando que regras fiquem espalhadas pelos Controllers.
DTO (Data Transfer Object)	Comunicação entre front-end e back-end.	Definir os dados recebidos e enviados pela API, evitando expor diretamente entidades e campos internos do banco.
Dependency Injection	Controllers, Services e Repositories gerenciados pelo Spring.	Reduzir acoplamento e facilitar substituição de dependências e testes.
MVC	Organização da camada web do back-end e integração com a interface.	Separar responsabilidades de requisição, processamento e

		apresentação dos dados retornados pela API.
Strategy (quando houver variação de canal)	Serviço de notificações, caso sejam implementados diferentes provedores ou canais.	Permitir variar o mecanismo de envio sem alterar a regra de negócio do agendamento. Será aplicado somente se houver mais de uma estratégia concreta.

Os padrões serão aplicados conforme a necessidade de cada módulo. Não serão adotados CQRS, Saga ou microsserviços na versão inicial, pois o projeto não possui, neste momento, requisitos que justifiquem a complexidade adicional desses padrões.

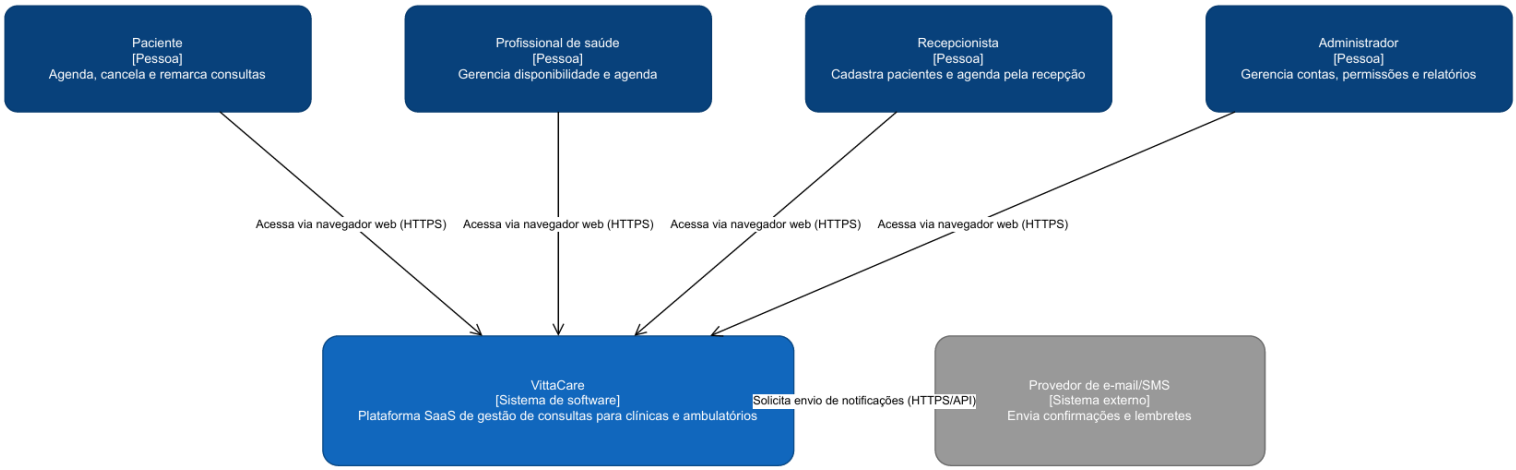
1.8. Modelo C4

O modelo C4 será utilizado para representar a arquitetura do VittaCare em quatro níveis de detalhamento: contexto, containers, componentes e código. Os diagramas devem manter consistência com o monólito modular, com a stack tecnológica e com a infraestrutura definida neste documento.

1.8.1. C1 — Diagrama de contexto

O diagrama de contexto apresenta o VittaCare como um sistema central e demonstra sua relação com os usuários e sistemas externos. Os atores são Paciente, Profissional de Saúde, Recepcionista e Administrador. O sistema externo previsto é o provedor de e-mail/SMS utilizado para notificações. Os usuários acessam o VittaCare por navegador web; o VittaCare envia solicitações de notificação ao provedor externo por HTTPS/API.

Diagrama C4 nível 1 — Contexto (VittaCare)
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014



1.8.2. C2 — Diagrama de containers

O diagrama de containers apresenta as principais unidades executáveis e os serviços de infraestrutura utilizados pelo VittaCare. Embora o back-end seja implantado em duas instâncias para disponibilidade, as duas instâncias executam a mesma aplicação monolítica modular; não representam microsserviços.

Navegador Web / React: executa a interface dos usuários e comunica-se com a API por HTTPS (porta 443), utilizando JSON.

Application Load Balancer (AWS): recebe o tráfego HTTPS público e distribui as requisições entre as instâncias EC2, utilizando health checks para identificar destinos não saudáveis.

EC2 A — aplicação VittaCare: executa Docker e a aplicação Java/Spring Boot, disponibilizando API REST internamente na porta configurada (por exemplo, 8080).

EC2 B — aplicação VittaCare: executa a mesma imagem e versão da aplicação Java/Spring Boot, também registrada como destino do ALB. As instâncias devem estar distribuídas em zonas de disponibilidade distintas, conforme disponibilidade regional e configuração contratada.

Amazon RDS PostgreSQL Multi-AZ: armazena os dados persistentes; a comunicação da aplicação com o banco ocorre pela porta TCP 5432, em rede privada e com criptografia em trânsito habilitada.

Amazon S3: armazena arquivos necessários à plataforma, sem utilizar o disco local das instâncias como armazenamento persistente compartilhado.

Amazon CloudWatch: recebe métricas, logs e alarmes dos recursos da aplicação e da infraestrutura.

Backups automatizados do RDS com point-in-time recovery (PITR): permitem restaurar o banco a um ponto no tempo, com logs de transação; a configuração deverá manter janela de recuperação compatível com RPO de até 15 minutos e retenção diária mínima de 7 dias.

AWS Backup: manterá snapshots semanais por pelo menos 4 semanas; o RTO de até 4 horas deverá ser comprovado por testes de restauração.

Amazon CloudFront: distribui conteúdo estático, quando utilizado, integrado ao S3.

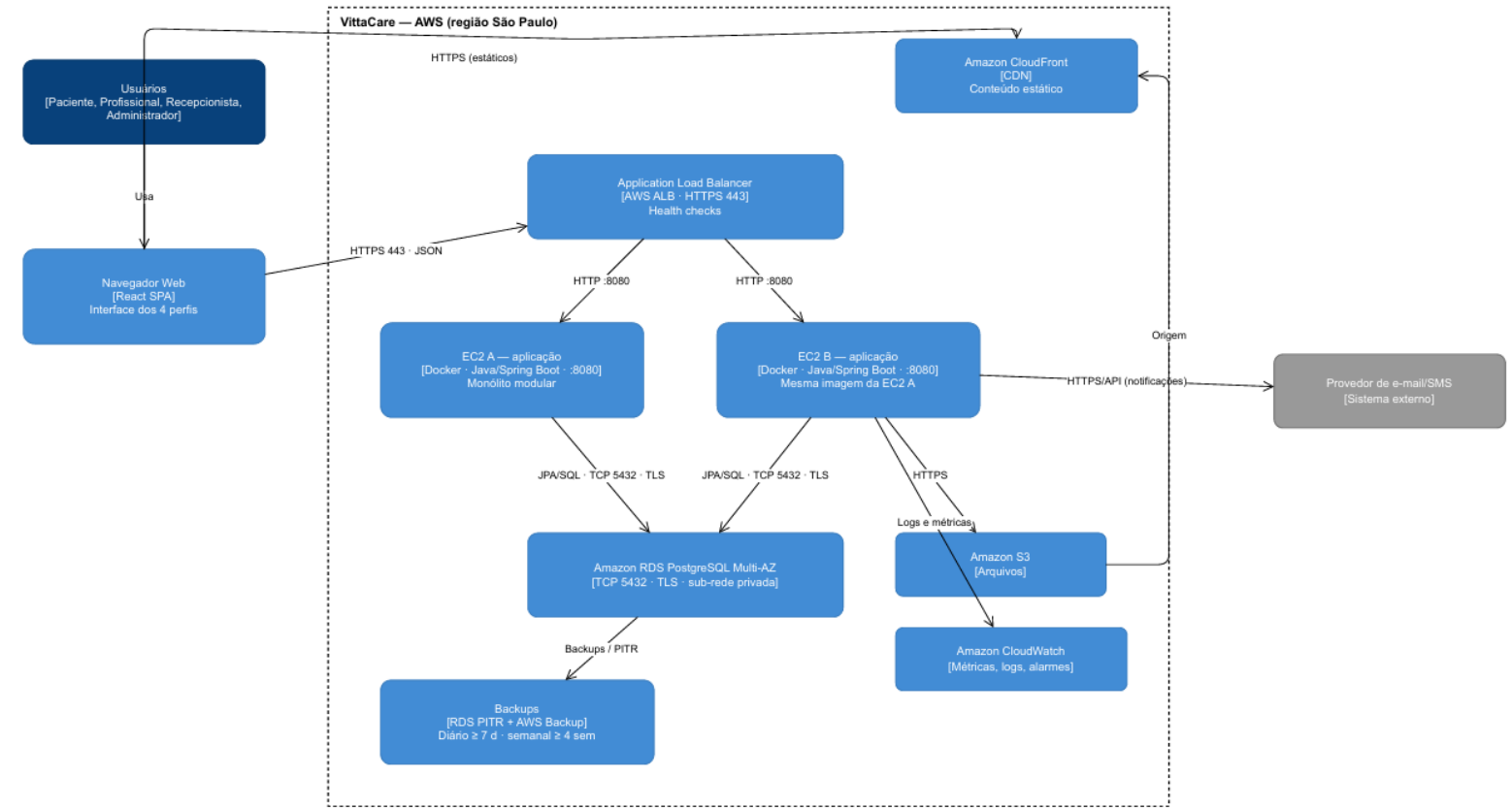
A política de recuperação deverá combinar point-in-time recovery (PITR) do RDS, com retenção diária mínima de 7 dias e logs de transação, e snapshots semanais gerenciados pelo AWS Backup com retenção mínima de 4 semanas. O RPO de até

15 minutos depende da configuração e disponibilidade dos backups contínuos; o RTO de até 4 horas deverá ser verificado por exercícios de restauração documentados. O backup diário isolado, por si só, não atende ao RPO de 15 minutos.

O ALB deverá encaminhar tráfego somente para instâncias saudáveis. As instâncias EC2 não deverão expor diretamente a porta da aplicação à internet; o acesso deverá ser permitido apenas a partir do Security Group do ALB. O banco RDS permanecerá em sub-redes privadas e aceitará conexões apenas das instâncias de aplicação autorizadas.

Os ambientes DEV, STAGING e PROD serão separados. O desenho de implantação deverá representar os recursos de produção com duas instâncias EC2 e ALB, e os ambientes não produtivos com recursos isolados e dimensionamento próprio. A topologia de produção é a referência para a disponibilidade definida no RNF006.

Diagrama C4 nível 2 — Containers (VittaCare)
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | ADR-001, ADR-007

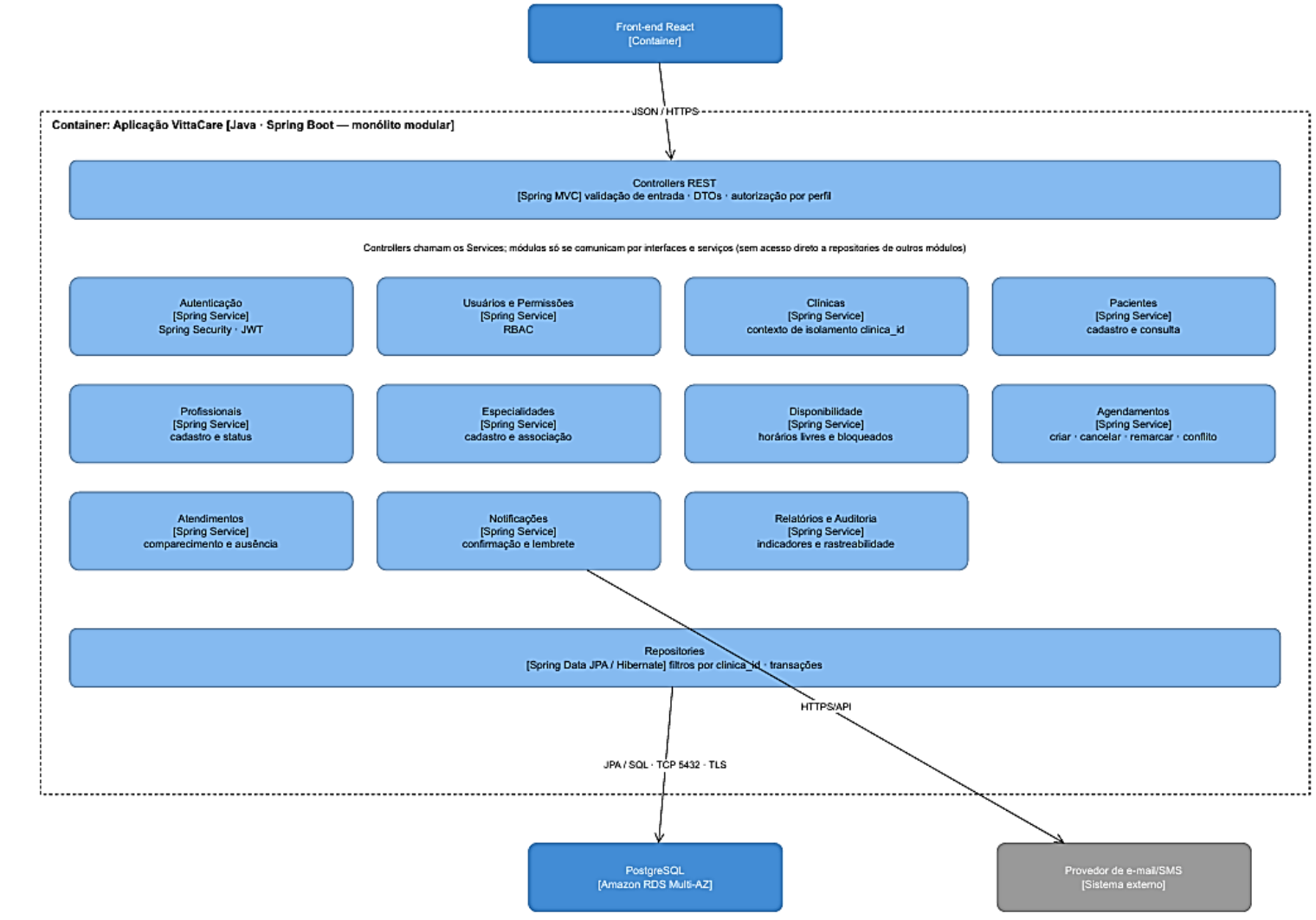


1.8.3. C3 — Diagrama de componentes

O diagrama de componentes detalha a organização interna da aplicação monolítica. O front-end React consome a API REST disponibilizada pelos Controllers. No back-end, os Controllers dependem dos Services; os Services aplicam as regras de negócio e utilizam os Repositories para persistência. Os módulos de autenticação, usuários e clínicas, pacientes, profissionais e especialidades, disponibilidade, agendamentos, atendimentos, notificações e relatórios/auditoria permanecem dentro do mesmo container de aplicação.

Os Repositories utilizam Spring Data JPA/Hibernate para acessar o PostgreSQL. O módulo de notificações integra-se ao provedor externo por HTTPS. As dependências entre módulos deverão ocorrer por interfaces e serviços internos, evitando acesso direto a estruturas internas de outros módulos.

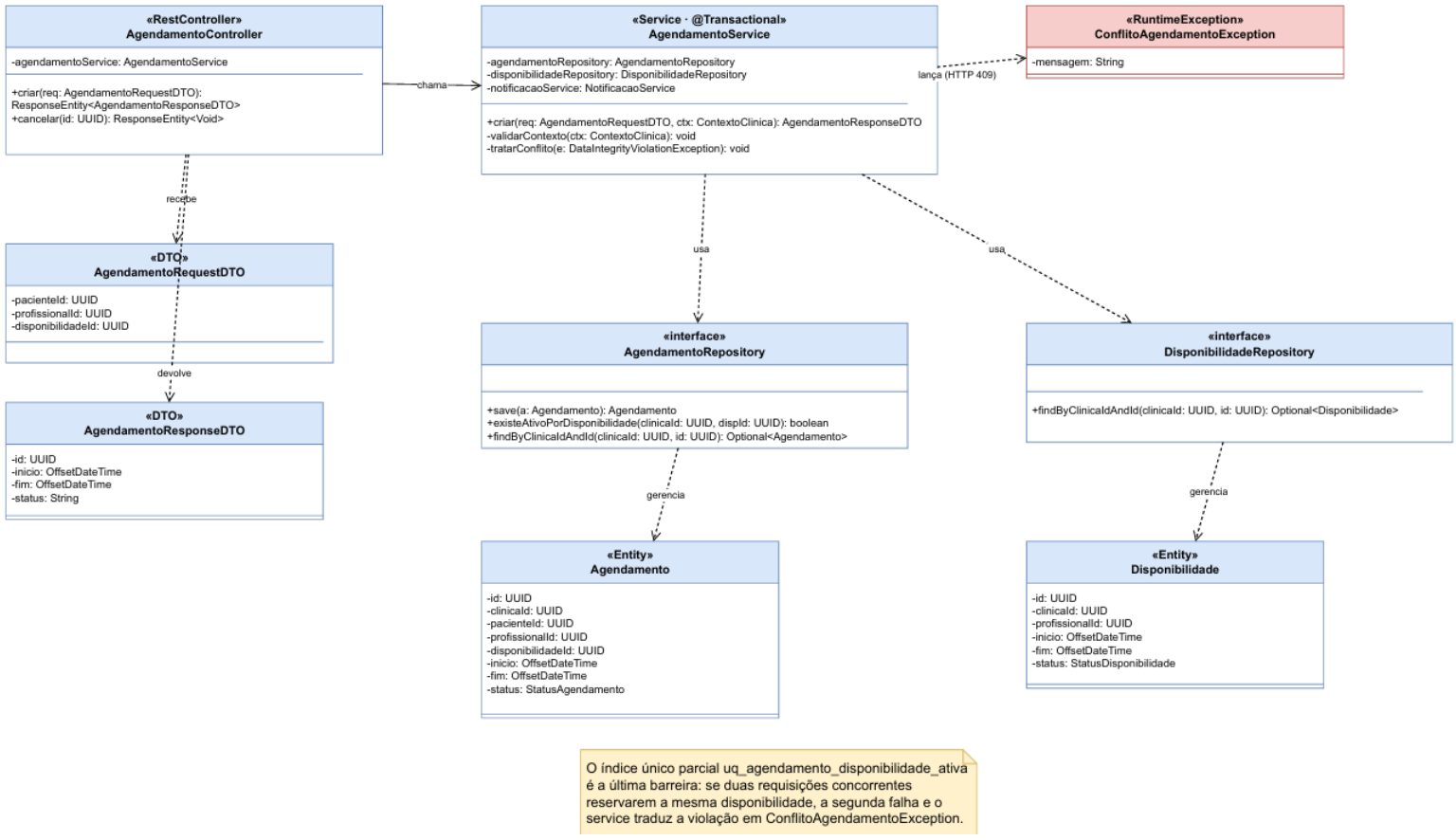
Diagrama C4 nível 3 — Componentes do container Aplicação Spring Boot
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | ADR-001, ADR-003, ADR-009



1.8.4. C4 — Diagrama de código

O diagrama de código detalhará um componente crítico: o serviço de agendamento. A estrutura proposta inclui AgendamentoController, AgendamentoService, DisponibilidadeRepository, AgendamentoRepository, entidades Agendamento e Disponibilidade, DTOs de entrada e saída e o mecanismo de transação do Spring. O AgendamentoService valida o contexto da clínica, verifica a disponibilidade, executa a criação em transação e trata conflitos de concorrência antes de retornar o resultado ao Controller.

Diagrama C4 nível 4 — Código do componente crítico: serviço de agendamento
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | RF014, RF018, RNF003, RNF008



1.9. Architecture Decision Records (ADRs)

Os ADRs registram as decisões arquiteturais relevantes, incluindo contexto, decisão, alternativas descartadas e consequências. As decisões abaixo deverão ser revisadas caso os requisitos ou as premissas de dimensionamento sejam alterados.

ADR-001 — Adoção da arquitetura de Monólito Modular

Contexto: o VittaCare possui módulos de negócio relacionados e uma equipe que precisa desenvolver e manter a primeira versão com complexidade operacional controlada.

Decisão: adotar monólito modular com separação em camadas e módulos de domínio.

Alternativas descartadas: microsserviços independentes, por exigirem implantação, observabilidade, comunicação distribuída e gestão de dados por serviço; monólito sem modularização, por dificultar a separação de responsabilidades.

Consequências: implantação e transações mais simples; será necessário impor limites entre módulos e evitar dependências circulares.

ADR-002 — Utilização do React no Front-end

Contexto: o sistema possui quatro perfis de usuário, com interfaces e fluxos distintos que consomem uma API REST, e a equipe prevista conta com um desenvolvedor front-end (Bloco 7). A interface precisa oferecer navegação por teclado e compatibilidade com Chrome, Firefox e Edge (RNF016 e RNF017).

Decisão: utilizar React para construir a interface baseada em componentes.

Alternativas descartadas: Angular, por ser um framework completo que impõe TypeScript, injeção de dependência e estrutura própria de módulos, aumentando a curva de aprendizado e o volume de código-base para um escopo de telas de cadastro, agenda e consulta e para uma equipe front-end de uma pessoa; páginas renderizadas exclusivamente no servidor, por não atenderem tão diretamente à interação de agenda e seleção de horários planejada.

Consequências: componentes reutilizáveis e integração desacoplada via API; será necessário padronizar rotas, estado, validação e acessibilidade, pois o React não impõe esses padrões.

ADR-003 — Utilização de Java e Spring Boot no Back-end

Contexto: o sistema exige regras de negócio, validações, autorização, transações e integração com PostgreSQL.

Decisão: utilizar Java com Spring Boot para implementar a API e os serviços.

Alternativas descartadas: Node.js com Express, por exigir composição manual de mais recursos estruturais e de segurança que já são integrados ao ecossistema Spring adotado.

Consequências: organização em camadas, injeção de dependência e integração com segurança e persistência; a equipe deverá manter testes e configuração padronizada.

ADR-004 — Utilização do PostgreSQL

Contexto: o domínio contém entidades relacionadas e regras de integridade, especialmente em agendamentos e no isolamento por clínica. A regra mais crítica (RF018 e RNF008) exige impedir dois agendamentos ativos para a mesma disponibilidade, mesmo com requisições concorrentes.

Decisão: utilizar PostgreSQL como banco relacional, hospedado no Amazon RDS.

Alternativas descartadas: MongoDB, por não oferecer o mesmo modelo relacional e o conjunto de constraints (FKs, FKs compostas, CHECK e UNIQUE) que atendem diretamente à integridade do domínio e ao isolamento por clinica_id. MySQL, por não oferecer índices únicos parciais nativos: o VittaCare depende de um índice único parcial (uq_agendamento_disponibilidade_ativa, restrito aos status AGENDADO e CONFIRMADO) como última barreira contra reservas duplicadas, e reproduzi-lo no MySQL exigiria soluções alternativas, como colunas geradas, aumentando a complexidade da regra mais crítica do sistema.

Consequências: uso de FKs, constraints, transações e índices; o modelo lógico inclui clinica_id e restrições adequadas ao isolamento multi-tenant; a equipe passa a depender de recursos específicos do PostgreSQL, como o índice parcial, o que deve ser considerado em eventual troca de banco.

ADR-005 — Comunicação por API REST e JSON

Contexto: front-end e back-end serão desenvolvidos separadamente e precisam de contratos explícitos.

Decisão: disponibilizar API REST com JSON sobre HTTPS.

Alternativas descartadas: GraphQL, pois as operações previstas podem ser representadas por recursos e endpoints REST; comunicação direta com o banco, por comprometer segurança e separação de camadas.

Consequências: contratos HTTP padronizados e integração simples; será necessário versionar e documentar endpoints.

ADR-006 — Autenticação com Spring Security e JWT

Contexto: os quatro perfis possuem permissões diferentes e o sistema é multi-tenant.

Decisão: utilizar Spring Security para autenticação/autorização e JWT assinado para identificar sessões de acesso à API.

Alternativas descartadas: autenticação implementada manualmente, por aumentar risco de falhas; sessão tradicional como único mecanismo, por não corresponder ao modelo de API stateless planejado.

Consequências: validação de assinatura, expiração, papéis e contexto de clínica em cada requisição protegida; tokens deverão ter validade e armazenamento seguros.

ADR-007 — Hospedagem na AWS com redundância da aplicação

Contexto: RNF006 define disponibilidade mensal mínima de 99,9%; o estudo de viabilidade estima duas EC2, ALB e RDS Multi-AZ.

Decisão: hospedar a aplicação em duas instâncias EC2 atrás de um Application Load Balancer, com RDS PostgreSQL Multi-AZ e recursos complementares AWS.

Alternativas descartadas: uma única EC2 sem balanceamento, por criar ponto único de falha na camada de aplicação; servidor físico próprio, por transferir à equipe a gestão de hardware e disponibilidade.

Consequências: distribuição de tráfego e health checks; o desenho e a operação deverão contemplar zonas de disponibilidade, monitoramento, custos de transferência e testes de failover. A topologia contribui para o RNF006, mas não garante sozinha 99,9%: a disponibilidade deverá ser medida ponta a ponta.

ADR-008 — Utilização de Docker

Contexto: os ambientes DEV, STAGING e PROD precisam executar a aplicação de maneira consistente.

Decisão: empacotar o back-end em containers Docker.

Alternativas descartadas: configuração manual independente em cada servidor, por aumentar diferenças entre ambientes.

Consequências: imagens versionadas e implantação repetível; será necessário manter imagens atualizadas, reduzir privilégios e proteger o processo de build.

ADR-009 — Utilização de Spring Data JPA e Hibernate

Contexto: a aplicação possui operações CRUD e relacionamentos entre entidades persistidas no PostgreSQL.

Decisão: utilizar Spring Data JPA e Hibernate como solução principal de persistência.

Alternativas descartadas: JDBC puro como padrão de todas as operações, por exigir maior volume de código repetitivo; consultas SQL manuais para todos os casos, por dificultar padronização.

Consequências: repositories e mapeamento objeto-relacional; consultas críticas deverão ser avaliadas quanto a índices, N+1, bloqueios e desempenho.

ADR-010 — Documentação da API com Swagger e OpenAPI

Contexto: o front-end depende de contratos claros para consumir endpoints e testar respostas.

Decisão: documentar a API com OpenAPI e disponibilizar visualização por ferramentas Swagger.

Alternativas descartadas: documentação exclusivamente manual, por aumentar o risco de divergência; ausência de documentação formal, por dificultar integração e testes.

Consequências: contratos consultáveis e padronizados; a documentação deverá ser atualizada junto às alterações da API.

ADR-011 — Versionamento com Git e GitHub

Contexto: o desenvolvimento será realizado em equipe e exige histórico, colaboração e revisão.

Decisão: utilizar Git para controle de versão e GitHub para hospedagem e colaboração.

Alternativas descartadas: compartilhamento manual de arquivos, por dificultar rastreabilidade e integração; controle de versão centralizado, por não ser a opção selecionada para o fluxo da equipe.

Consequências: uso de branches, commits e pull requests; será necessário definir convenções e regras de revisão.

ADR-012 — Isolamento Lógico dos Dados por Clínica

Contexto: o VittaCare será SaaS multi-tenant e não poderá permitir acesso cruzado aos dados de clínicas diferentes.

Decisão: utilizar banco compartilhado com isolamento lógico por clinica_id, validado no back-end em todas as operações e reforçado por constraints relacionais.

Alternativas descartadas: banco sem identificação de organização, por não separar os registros; banco independente por clínica, por aumentar a complexidade operacional e de custos na versão inicial.

Consequências: consultas e operações deverão aplicar escopo de clínica; serão obrigatórios testes de acesso cruzado e auditoria das operações sensíveis.

1.10. Considerações finais

A arquitetura proposta para o VittaCare busca equilibrar organização, segurança, disponibilidade, manutenção e possibilidade de evolução. A adoção de um monólito modular permite separar responsabilidades sem introduzir, na versão inicial, a complexidade de uma arquitetura distribuída.

A utilização de React, Java com Spring Boot e PostgreSQL estabelece uma base compatível com a aplicação web e com as regras de integridade do domínio. A infraestrutura AWS proposta — duas instâncias EC2 atrás de um ALB e banco RDS PostgreSQL Multi-AZ — está alinhada ao objetivo de disponibilidade, enquanto CloudWatch e a política de backup apoiam monitoramento e recuperação.

Os requisitos de 99,9% de disponibilidade, 1.000 usuários simultâneos, RPO de até 15 minutos, RTO de até 4 horas e isolamento lógico entre clínicas deverão ser verificados por testes, monitoramento e configuração efetiva da infraestrutura. A arquitetura representa uma proposta técnica; a comprovação desses níveis dependerá de validação operacional antes da entrada em produção.